

Independent audit of the Fable v0.2.1 handoff

19 September 2026 | Taleb Lean formalization | Review of the delivered archive

Assessment

Accept the existing encoded mathematical results as a sound, reproducible foundation under the project's stated definitions and standard Lean axioms. Request a focused corrective pass on the verification harness and source-review documentation before using this package as the acceptance gate for further formalization.

The current proofs pass. The remaining engineering concern is that the verifier can certify some deliberately defective future changes. This distinction matters: the fault injections below were made only in a disposable audit copy, and none is present in the received archive.

Fable made useful, restrained changes. The complete patch reproduces the delivered tree exactly. No existing theorem statement, proof body, definition, instance, or alias was changed. The two added theorems document boundary behavior; they do not discharge two additional substantive book obligations.

Independently reproduced evidence

Check	Independent result
Archive checksum	Matches the supplied SHA-256
Internal manifest	All 121 entries match; 122 files including the manifest
Patch replay	Applies to v0.2.0 and reproduces all 122 final files byte-for-byte
Existing source changes	Two diagnostic theorems added; one comment corrected; existing proofs unchanged
Clean project build	Exit 0; 24 project modules built; 2643 jobs; no warnings or errors
Delivered verification script	Exit 0; 52 theorems, 1 instance, 16 aliases
Reproduced verification record	Identical to the supplied record except elapsed time
Reproduced environment inventory	Identical to the supplied inventory
Expanded axiom audit	All 137 constants in imported proof modules, including 35 internal constants, stay within the allowlist
Backlog consistency	All 158 JSON titles, targets, and gap descriptions agree with their Markdown entries

The clean build took 47.80 seconds; the delivered verifier took 114.68 seconds in this environment. Lean was 4.24.0, compiler commit 797c613eb9b6d4ec95db23e00af9ac6657f24b. Mathlib was f897ebcf72cd16f89ab4577d0c826cd14afaafc7. All nine dependency revisions were checked, with no tracked source modifications, before reusing their caches. The project itself started with no build outputs.

The permitted axiom set was {propext, Classical.choice, Quot.sound}. None of the 137 scanned constants had an additional axiom dependency. Some internal constants require fewer axioms. The figure 69 includes aliases, diagnostics, and supporting lemmas; it is not a count of independently formalized book theorems.

The original ZIP and preserved extraction were not modified. Executions and mutations used separate working copies. This audit did not re-run the original sixteen pre-repair files; their historical evidence was checked for preservation and consistency.

A1. High priority: private unfinished proofs can escape the verifier

Location: scripts/FableInventory.lean, the n.isInternalDetail branch in run; scripts/verify.py, declaration discovery and the build/axiom checks.

The environment inventory counts internal names but does not collect their axiom dependencies. Lean gives user-written private declarations internal names. The textual declaration scanner also skips private theorem. Finally, the verifier inspects diagnostics from the later axiom-printing run, while accepting build output containing a warning about an unused private proof.

The audit fixture appends this to AuditRepairs/RegularVariation.lean:

```
private theorem audit_private_gap : False := by sorry
#print axioms audit_private_gap
```

Observed: verifier exit 0 and status PASS. The build log explicitly reports that the private theorem uses sorry and depends on [sorryAx]. The inventory lists 102 public/generated constants and counts 36 internal names, but omits the injected private theorem from its axiom-checked rows. Its public proof count remains 69. Evidence: probes/private_sorry/.

This does not refute any delivered theorem. It refutes the verifier's ability to enforce the promised absence of unproved project declarations as the project grows. The new environment scan fixes public discovery cases but leaves this original class of omission unresolved.

Required repair: perform axiom checks for every project constant before any visibility or generated-name filtering. Keep public API counts separate from the complete trust scan. Add a regression that requires a nonzero process exit and a FAIL record for an unused private theorem or definition containing sorry, and for a private axiom. Check relevant build diagnostics as an additional safeguard.

A2. Medium priority: namespace membership is mistaken for generated provenance

Location: `scripts/FableInventory.lean`, `generatedByInductive` assignment; `scripts/verify.py`, `env_checked` computation.

The inventory labels a constant as generated when its parent or grandparent namespace names a project inductive type. A user can legitimately write a theorem inside that namespace. That theorem is then exempted from the user-proof inventory comparison even though Lean did not generate it.

Audit fixture, appended to `AuditRepairs/Stable.lean`:

```
protected theorem StableAudit.StableParameters.audit_user_theorem : True := True.intro
```

Observed: verifier exit 0 and status PASS. The environment inventory lists the new theorem and incorrectly sets `generatedByInductive=true`. Listed constants increase to 103, while the user-proof count remains 69. Evidence: `probes/structure_namespace/`.

The axiom scan still examines this public constant, so this issue alone does not conceal an extra axiom. It breaks the claimed reconciliation of all user-written theorem/instance declarations and can silently undercount an exported result.

Required repair: use trustworthy declaration provenance where available, or a precise, reviewed classification of actual generated declarations. Do not infer generated status merely from a namespace prefix. Include a user-written theorem under a structure namespace in the regression suite.

A3. Medium priority: Python optimization disables acceptance checks

Location: `scripts/verify.py`, assertions enforcing toolchain/dependency pins, inventory equality, axiom closure, and alias correspondence.

Python removes `assert` statements under `python3 -O` or `PYTHONOPTIMIZE=1`. These statements currently carry nearly every acceptance condition. The report then writes success booleans unconditionally after the omitted checks.

The audit changes Proof16's documented target to `AuditProbability.charFun_convolutionPower`, which disagrees with its actual alias, and runs the verifier in ordinary and optimized modes.

Observed: ordinary execution rejects the wrong map with exit 1 and FAIL; optimized execution returns exit 0 and PASS. In the optimized run, the report even states `alias_targets_match_replacement_map=true` despite the deliberately incorrect target. Evidence: `probes/alias_control/` and `probes/optimized_alias/`.

This design weakness was inherited from the original handoff and was also used for the new checks. It does not undermine the ordinary-mode results reproduced above.

Required repair: replace acceptance assertions with explicit checks that raise a dedicated verification exception, or explicitly reject optimized execution before doing any work. Compute report booleans from validated conditions. Test ordinary execution and `PYTHONOPTIMIZE=1` against the same deliberately invalid fixture.

A4. Low priority: one negative-test record reports a successful exit

Location: `evidence/fable/final/harness_negative_tests.log`, Test 2.

The supplied log displays an inventory-mismatch message followed by `exit=0`, whereas the surrounding documents say both rejection tests failed as intended. It does not record enough command context to determine whether zero belongs to a wrapper or to the verifier.

The independent control in this audit, an ordinary public protected theorem outside a structure namespace, correctly yields exit 1 and FAIL with an environment/regex mismatch. The corresponding rejection mechanism works; the supplied historical exit-status record is ambiguous. Evidence: `probes/protected_control/`.

Required repair: preserve the original log and add a fresh rejection-test record with the exact command, verifier subprocess exit code, expected exit behavior, and resulting `verification.json`. A wrapper may exit zero when a rejection test succeeds, but label its status separately from the verifier's exit status.

Additional hardening: the verifier checks dependency HEAD revisions but not working-tree modifications. This audit checked tracked dependency changes separately. Add that check, or describe the pin assurance narrowly. No dirty dependency was found in this reproduction.

Source assessment: Fable's two new gates are correct

Both findings were confirmed directly on the rendered pinned book PDF:

- **G17, printed p. 282 / PDF p. 296:** the parentheses in both displayed S1 branches are misplaced. For positive scale, the literal expression can have modulus greater than one. The project's S1 expression uses the standard placement and agrees with Nolan's Definition 1.8, equation (1.6). The $\alpha = 1$ logarithmic branch and the location parameter for convolution powers are consistent.
- **G18, printed p. 99 / PDF p. 113:** the displayed limit of $\log(w_1 x^{(-a_1)} + w_2 x^{(-a_2)}) / \log(x)$ needs the value $-a_2$, under the stated positive-tail conditions. The finite-exponent API uses the corresponding negative-log convention correctly.

These are documentation/source findings. The delivered Lean statements do not use the erroneous printed formulas literally.

M1. Qualify the claim that Property 5.1 is unaffected

Fable correctly repairs the sign, but its review says Property 5.1 itself is unaffected. Read as approval of the property's full printed scope, that is too strong. The source explicitly permits dependence and does not require nonnegative summands. Positive weights alone do not prevent cancellation.

Counterexample: let Z have a Pareto tail $P(Z > x) = x^{(-a)}$ for $x \geq 1$, with $a > 0$. Let an independent sign E take values $+1$ and -1 with equal probability. Set $X = E \cdot Z$ and $Y = -2 \cdot E \cdot Z$. Their right tails have the same finite exponent a , and the laws are different. With strictly positive weights 2 and 1, $2 \cdot X + Y = 0$ identically. The sum has zero right tail for positive thresholds, so it does not retain finite exponent a .

The existing T029 backlog already mentions cancellation for signed dependent laws. Make this caveat explicit in G18 and the review, and classify the unrestricted source statement accordingly. The corrected two-power analytic limit does not establish the unrestricted probabilistic statement.

A useful valid target is the finite sum of nonnegative variables with positive weights. For n positive, event inclusions give

$$\begin{aligned} \max_i P(w_i X_i > x) \\ &\leq P(\sum_i w_i X_i > x) \\ &\leq \sum_i P(w_i X_i > x/n). \end{aligned}$$

These bounds avoid independence and support a minimum finite logarithmic tail-exponent result under appropriate tail-limit assumptions. Exact convolution-tail asymptotics are a separate, stronger target. The counterexample and bounds here are mathematical review arguments, not new Lean proofs delivered by this audit.

M2. Add a source gate separating regular variation from exact stability

On the same printed p. 282 inspected for G17, the paragraph below the characteristic function identifies a regularly varying random variable of order α with an α -stable random variable. That identification is false. Regular variation describes tails; stability is an exact distributional property involving independent sums.

For example, a Pareto law with $\alpha = 3/2$ has a regularly varying tail and support $[1, \infty)$. A nondegenerate $3/2$ -stable law has support on the entire real line, by Nolan's Lemma 1.10. Thus the Pareto law itself is not $3/2$ -stable. Appropriate normalized sums can converge to a stable law; that is the domain-of-attraction statement.

T093 already targets a domain-of-attraction theorem with tail balance and normalization, which is the appropriate direction. Add an explicit source-check note for the erroneous identification rather than inheriting it into future theorem statements. No current delivered theorem makes this identification.

Minor documentation correction: Fable's Proof01 row says strict positivity of $\log$ holds on a closed domain starting at $x_{\min} \geq 1$. At $x_{\min} = 1$, $\log$ is zero. Use $x_{\min} > 1$, an open lower endpoint, or eventual positivity. The Lean theorem is unaffected.

Per-entry assessment of the existing formal statements

All sixteen aliases compile and were included in the axiom review. The following limits concern interpretation, not rejected Lean proofs.

Entry	Assessment	Remaining interpretation boundary
01	Accept	Slowly varying log in the ratio-only API; eventual positivity is a separate fact.
02	Accept	Nonzero constants; the book's positive class excludes negative constants.
03	Accept	Nonzero finite-limit implication; no converse claim.
04	Accept	Products of functions add indices; this is not a product-of-random-variables theorem.

Entry	Assessment	Remaining interpretation boundary
05	Accept	Real-power closure with eventual positivity.
06	Accept	Ratio representation; no Karamata integral representation.
07	Accept	Finite logarithmic exponent for an analytic power tail; not a constructed Pareto law.
08	Accept	Conditional preservation under self-convolution of a subexponential law.
09	Accept	Convexity on the correct whole-real domain; stochastic Jensen remains separate.
10	Accept	Piecewise Frechet formula and positive sample size; measure/iid maximum theorem open.
11	Accept	Gumbel formula identity with positive sample size.
12	Accept	Gaussian S1 expression; variance is twice the squared scale.
13	Accept	Gaussian exponential identity; law identification remains separate.
14	Accept	Centered Cauchy S1 expression for nonnegative scale, including zero scale.
15	Accept	Cauchy exponential identity.
16	Accept conditionally	Convolution-law identification assumes characteristic-function premises; general stable-law existence remains open.

Recommended next pass

- 1. Close the verification gaps first.** Scan all constants for axiom dependencies and verify that every shipped proof module is imported; correct generated-declaration classification; make acceptance checks survive Python optimization; regenerate unambiguous negative-test evidence. Preserve theorem statements and toolchain pins. Require the defective fixtures above to be rejected, the valid package to pass, and the final archive to reproduce from a fresh extraction.
- 2. Update source gates and documentation.** Retain G17/G18, add the cancellation counterexample and regular-variation/stability distinction, and synchronize T029/T093 with those qualifications.
- 3. Select one mathematics milestone.** Fable's two-term power-tail limit is a reasonable small task. The Gaussian instantiation is particularly useful because it supplies a concrete family satisfying Proof16's hypotheses. Prove the characteristic-function identification with variance $2 \cdot \text{scale}^2$, including zero scale, and exercise the convolution bridge. Treat this primarily as project integration using existing Mathlib infrastructure.
- 4. For substantive Mathlib extension, choose a separate API target.** A positive/measurable regular-variation interface is a plausible candidate, subject to checking current upstream definitions and contribution requirements. The pinned snapshot and this audit do not establish novelty. Keep the large stable-existence and full EVT programs as explicit later milestones.

The larger unresolved items remain stable-law existence, concrete subexponential laws, probability-measure realizations of the EVT formulas, positive/measurable regular variation and its deeper theorems, moment integrals, and the other inventory families. The 158-entry backlog is not an exhaustive, fully checked atomization of the book.

Inputs needed

No additional input is needed to assess this handoff. The two README attachments serve different purposes: one is the outer deliverables guide, and the other matches the project README. The standalone review and changelog match the copies inside the ZIP. The patch and auxiliary validation log were useful for provenance and reproduction.

Work on the quantum/classical application requires its actual repository or source archive, a representative input and expected output, the intended execution backend, and a concrete office-demo acceptance criterion. This formalization package does not contain that application. No conclusion about its runtime readiness or the separate paper's empirical claim follows from these Lean results.

Provenance, limits, and references

Reviewed archive: `Taleb_Lean_Implementation_Handoff_v0.2.1_fable.zip`.

`c492cc14c470f8cd3d237f884a4c3157195a11c1d5e56e62c52a818f9053008a`

Previous handoff used for exact patch replay:

`8a8fb28af0b492c34edd02275ace60b213eedac1f41ec2514505588157cab406`

Pinned book: Taleb, Statistical Consequences of Fat Tails, arXiv:2001.10488v4, 523 PDF pages. Its local SHA-256 matches `758e18b7337840104db93296144d67cf5514bf2de128fe557dc84bc32a0ad567`. Printed Arabic page numbers differ from PDF page numbers by 14.

- [Taleb's pinned source](#): printed pp. 99 and 282 were rendered and visually inspected in this pass.
- [Nolan, Stable Distributions, Chapter 1](#): Definition 1.8 / equation (1.6), Lemma 1.10, and Section 1.8 support the parameterization, support, and domain-of-attraction distinctions above.
- Local authoritative evidence: the received archive, complete patch, pinned Lean/Mathlib sources, fresh command logs, expanded inventory, and fault-injection records in the companion evidence ZIP.

This is an audit of the delivered project and the source claims relevant to its changes, with review of all sixteen formal entry points. It is not a new line-by-line audit of all 523 book pages, an upstream novelty review, or a certification of the application. The diagnostic inventory variant extends Fable's collector to include internal constants; it uses Lean's own axiom collector rather than an independent implementation of the Lean kernel.